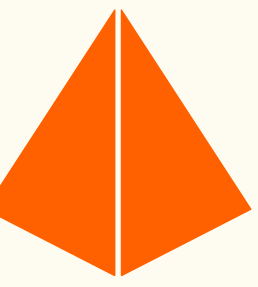


LeMauvaisCoin

Présentation globale - SQL

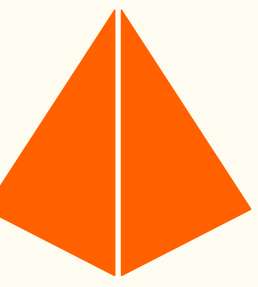
Maxime Richard / Alexandre MULARD



Sommaire

- Modélisation et Schéma
- Requêtes complexes et Transactions
- Performance et Optimisation
- Présentation du site
- Conclusion

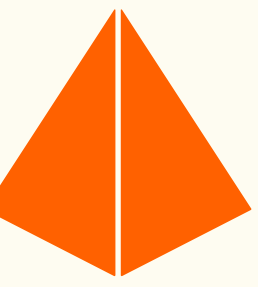
Modélisation et Schéma

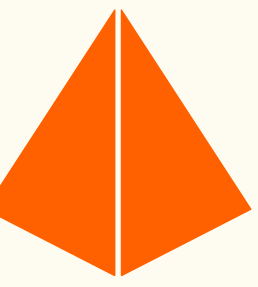


Relation de la base de donnée :

1-N	N-N
category ↔ product	user ↔ type
user ↔ sale_ad	command ↔ sale_ad
product ↔ sale_ad	
user ↔ command	
sale_ad ↔ picture	

Modélisation et Schéma

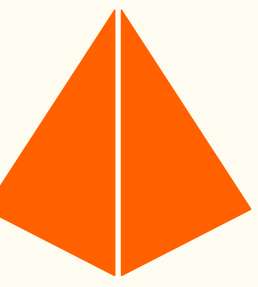




Modélisation et Schéma

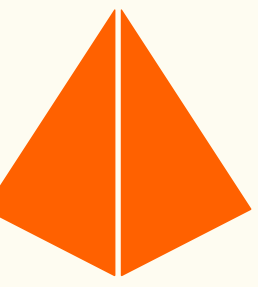
- Entités : User / Type / User_Type / Category / Product / Sale_id / Picture / Command / Command_product
- Normalisation (3NF) :
 - 1 seule valeur
 - 1 seul concept
 - Ne pas dépendre d'une autre colonne qui n'est pas la clé

Requêtes complexes et Transactions



Requête complexe (Produit le plus vendu) :

```
SELECT
    p.name AS nom_produit,
    cat.name AS categorie,
    COUNT(cp.sale_ad_id) AS nombre_de_ventes
FROM product p
JOIN category cat ON p.category_id = cat.id
JOIN sale_ad sa ON p.id = sa.product_id
JOIN command_product cp ON sa.id = cp.sale_ad_id
GROUP BY p.id, p.name, cat.name
ORDER BY nombre_de_ventes DESC
LIMIT 1;
```

Requêtes complexes et Transactions

Transaction :

```
BEGIN;
```

```
INSERT INTO command (date, user_id)
```

```
VALUES (CURRENT_TIMESTAMP, 3);
```

```
INSERT INTO command_product (command_id, sale_ad_id)
```

```
VALUES (LASTVAL(), 3);
```

```
UPDATE sale_ad
```

```
SET quantity = quantity - 1
```

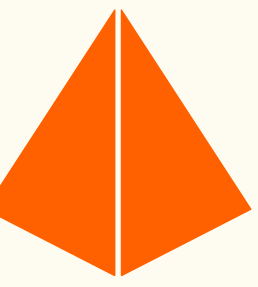
```
WHERE id = 3;
```

Si tout est OK :

```
COMMIT;
```

Si une erreur survient (ex: stock < 0), Postgres permet de faire :

```
ROLLBACK;
```



Performance et Optimisation

- Diagnostic : CA - EXPLAIN QUERY PLAN
- Optimisation : INDEX sur sale_id
- Recherche textuelle : INDEX GIN (mot-clé)

Alex-hexa's OrgFREE / SQL BDD / mainPRODUCTION Connect

FeedbackSearch... K ? ! [] [] []

SQL Editor

SQL Users Ranked by Revenue +

1 EXPLAIN SELECT u.firstname, u.lastname, SUM(sa.price)

2 FROM "user" u

3 JOIN sale_ad sa ON u.id = sa.user_id

4 JOIN command_product cp ON sa.id = cp.sale_ad_id

5 GROUP BY u.id, u.firstname, u.lastname;

< Search queries... +

> SHARED

> FAVORITES

> PRIVATE (1)

SQL Users Ranked by Revenue

> COMMUNITY

Templates

Quickstarts

View running queries

Results Explain Chart Export >

QUERY PLAN

HashAggregate (cost=80.35..81.10 rows=60 width=472)

Group Key: u.id

-> Hash Join (cost=24.27..69.05 rows=2260 width=456)

Hash Cond: (sa.user_id = u.id)

-> Hash Join (cost=12.93..51.58 rows=2260 width=20)

Hash Cond: (cp.sale_ad_id = sa.id)

-> Seq Scan on command_product cp (cost=0.00..32.60 rows=2260 width=4)

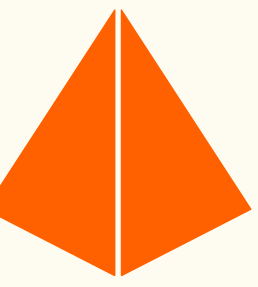
-> Hash (cost=11.30..11.30 rows=130 width=24)

-> Seq Scan on sale_ad sa (cost=0.00..11.30 rows=130 width=24)

-> Hash (cost=10.60..10.60 rows=60 width=440)

-> Seq Scan on "user" u (cost=0.00..10.60 rows=60 width=440)

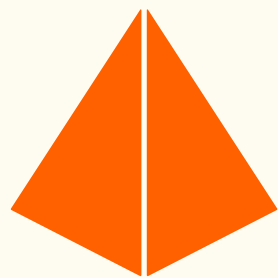
11 rows



Performance et Optimisation

- `CREATE INDEX idx_cp_sale_ad ON command_product(sale_ad_id);`

Performance et Optimisation



Alex-hexa's Org

FREE

SQL BDD

main

PRODUCTION

Connect

Feedback

Search... ⌘K

SQL Editor

Search queries...

+

> SHARED

> FAVORITES

PRIVATE (2)

SQL

Index on command_product.sa...

SQL

Users Ranked by Revenue

COMMUNITY

Templates

Quickstarts

View running queries

SQL

Users Ranked by Revenue

SQL

Index on command_product.sale_ad_id

+

1

EXPLAIN SELECT u.firstname, u.lastname, SUM(sa.price)

2

FROM "user" u

3

JOIN sale_ad sa ON u.id = sa.user_id

4

JOIN command_product cp ON sa.id = cp.sale_ad_id

5

GROUP BY u.id, u.firstname, u.lastname;

Results

Explain

Chart

Export

Source

Primary Database

Role postgres

Run ⌘↵

QUERY PLAN

GroupAggregate (cost=9.40..9.48 rows=4 width=472)

Group Key: u.id

-> Sort (cost=9.40..9.41 rows=4 width=456)

Sort Key: u.id

-> Nested Loop (cost=0.29..9.36 rows=4 width=456)

-> Nested Loop (cost=0.14..8.32 rows=4 width=20)

-> Seq Scan on command_product cp (cost=0.00..1.04 rows=4 width=4)

-> Index Scan using sale_ad_pkey on sale_ad sa (cost=0.14..1.81 rows=1 width=24)

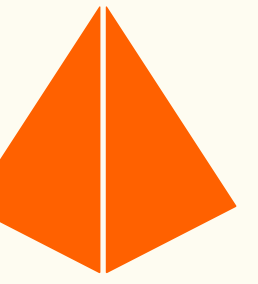
Index Cond: (id = cp.sale_ad_id)

-> Index Scan using user_pkey on "user" u (cost=0.14..0.26 rows=1 width=440)

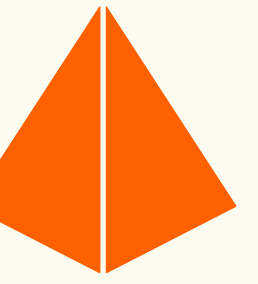
Index Cond: (id = sa.user_id)

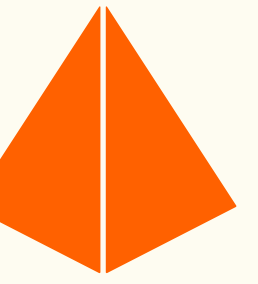
11 rows

Présentation

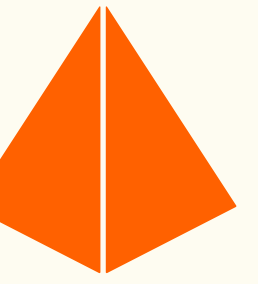


Conclusion





Merci pour votre écoute



Avez-vous des questions ?